

CHƯƠNG 09: SUY DIỄN TRONG LOGIC BẬC NHẤT

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 9

- ◇ 9.1 Từ Mệnh đề đến Bậc nhất (Propositional vs First-Order)
- ◇ 9.2 Hợp nhất và Cụ thể hóa (Unification & Lifting)
- ◇ 9.3 Suy diễn Liên kết Thuận (Forward Chaining)
- ◇ 9.4 Suy diễn Liên kết Ngược (Backward Chaining)
- ◇ 9.5 Phân giải Logic Bậc nhất (Resolution)

9.1 Từ Mệnh đề đến Logic Bậc nhất

◇ Ở chương trước, ta đã biết cách dùng bảng chân trị và thuật toán Phân giải để chứng minh tự động trong Logic Mệnh đề.

◇ **Vấn đề:** Trong Logic Bậc Nhất (FOL) có chứa Lượng từ ($\forall$, $\exists$). Làm sao để áp dụng các thuật toán cũ lên FOL?

◇ **Giải pháp ngây thơ (Propositionalization):**

- Ý tưởng là khử hết các Lượng từ để đưa FOL về lại Logic Mệnh đề.
- **Khử Lượng từ Phổ quát ($\forall$):** Thay thế biến bằng mọi hằng số có thể. (Ví dụ: Từ $\forall x \text{ King}(x)$, ta đẻ ra $\text{King}(\text{John}) \wedge \text{King}(\text{Richard})$).
- **Khử Lượng từ Tồn tại ($\exists$):** Đưa vào hằng số mới tinh (Hằng số Skolem). (Ví dụ: $\exists x \text{ Crown}(x)$ biến thành $\text{Crown}(C_1)$ với C_1 là tên của vương miện).

◇ **Điểm yếu chí mạng:** Nếu Cơ sở tri thức có vô hạn đối tượng (VD: Các con số), thao tác khử lượng từ sẽ tạo ra số lượng phương trình mệnh đề vô tận $\Rightarrow$ Tràn bộ nhớ!

9.2 Hợp nhất (Unification)

◇ Thay vì tạo ra hàng triệu mệnh đề, ta có thể **Làm khớp (Unify)** các biến để chỉ rút ra những kết luận thực sự cần thiết.

◇ **Thuật toán Hợp nhất** $\text{Unify}(p, q) = \theta$:

- Tìm một phép thế θ (Substitution) sao cho sau khi áp dụng, hai biểu thức p và q trở nên giống hệt nhau.

◇ **Ví dụ Hợp nhất:**

Giả sử có tri thức: "Biết cha của x là ai, thì x quen người đó"

$\Rightarrow \text{Knows}(x, \text{FatherOf}(x))$.

Câu hỏi truy vấn: "John quen ai?" $\Rightarrow \text{Knows}(\text{John}, y)$.

- Lời gọi: $\text{Unify}(\text{Knows}(\text{John}, y), \text{Knows}(x, \text{FatherOf}(x)))$
- Kết quả phép thế θ : $\{x/\text{John}, y/\text{FatherOf}(\text{John})\}$
- $\Rightarrow$ Máy tính tự động trả lời: John quen Cha của John!

Đồ thị Hợp nhất

```
function UNIFY( $x, y, \theta = \text{empty}$ ) returns a substitution to make  $x$  and  $y$  identical, or failure  
  if  $\theta = \text{failure}$  then return failure  
  else if  $x = y$  then return  $\theta$   
  else if VARIABLE?( $x$ ) then return UNIFY-VAR( $x, y, \theta$ )  
  else if VARIABLE?( $y$ ) then return UNIFY-VAR( $y, x, \theta$ )  
  else if COMPOUND?( $x$ ) and COMPOUND?( $y$ ) then  
    return UNIFY(ARGS( $x$ ), ARGS( $y$ ), UNIFY(OP( $x$ ), OP( $y$ ),  $\theta$ ))  
  else if LIST?( $x$ ) and LIST?( $y$ ) then  
    return UNIFY(REST( $x$ ), REST( $y$ ), UNIFY(FIRST( $x$ ), FIRST( $y$ ),  $\theta$ ))  
  else return failure  
  
function UNIFY-VAR( $var, x, \theta$ ) returns a substitution  
  if  $\{var/val\} \in \theta$  for some  $val$  then return UNIFY( $val, x, \theta$ )  
  else if  $\{x/val\} \in \theta$  for some  $val$  then return UNIFY( $var, val, \theta$ )  
  else if OCCUR-CHECK?( $var, x$ ) then return failure  
  else return add  $\{var/x\}$  to  $\theta$ 
```

Figure 1: Mô phỏng đồ thị thuật toán Hợp nhất (Unification) để tìm ra phép thế khớp nhất giữa hai biểu thức FOL.

9.3 Suy diễn Liên kết Thuận (Forward Chaining)

◇ **Ý tưởng cơ bản:** "Bắn súng không cần đích".

- Bắt đầu từ những dữ kiện có sẵn trong KB.
- Liên tục áp dụng các luật (Rules) có vế trái khớp với dữ kiện để suy ra dữ kiện mới (Vế phải).
- Lặp lại đến khi không thể suy ra gì thêm, hoặc đến khi sinh ra được câu trả lời.

◇ **Đặc điểm của Forward Chaining:**

- Là thuật toán **hướng dữ liệu (Data-driven)**. Thường dùng trong các hệ thống giám sát sự kiện thực tế, nơi dữ liệu mới liên tục ập đến.
- Ví dụ: Phát hiện nhiệt độ tang $\Rightarrow$ Báo cháy $\Rightarrow$ Kích hoạt vòi phun nước.

Mô phỏng Liên kết Thuận

```
function FOL-FC-ASK( $KB, \alpha$ ) returns a substitution or false
inputs:  $KB$ , the knowledge base, a set of first-order definite clauses
          $\alpha$ , the query, an atomic sentence

while true do
   $new \leftarrow \{\}$  // The set of new sentences inferred on each iteration
  for each rule in  $KB$  do
     $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$ 
    for each  $\theta$  such that  $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$ 
      for some  $p'_1, \dots, p'_n$  in  $KB$ 
         $q' \leftarrow \text{SUBST}(\theta, q)$ 
        if  $q'$  does not unify with some sentence already in  $KB$  or  $new$  then
          add  $q'$  to  $new$ 
           $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
          if  $\phi$  is not failure then return  $\phi$ 
  if  $new = \{\}$  then return false
  add  $new$  to  $KB$ 
```

Figure 2: Liên kết thuận: Bắt đầu từ luật và dữ kiện cơ sở, phát triển dần dần thành Cây suy diễn cho đến khi bao phủ toàn bộ.

9.4 Suy diễn Liên kết Ngược (Backward Chaining)

◇ **Ý tưởng cơ bản:** "Chạy ngược từ Đích".

- Bắt đầu từ **Câu truy vấn đích (Goal)**.
- Tìm các luật trong KB có Vế phải khớp với Goal.
- Lấy Vế trái của luật đó làm các Goal con (Sub-goals) mới và tiếp tục tìm kiếm.
- Dừng lại khi tất cả Sub-goals đều là những dữ kiện thực tế có sẵn.

◇ **Đặc điểm của Backward Chaining:**

- Là thuật toán **hướng mục tiêu (Goal-directed)**. Chỉ tìm kiếm những gì thật sự cần thiết cho đích đến, không lan man.
- **Ứng dụng cực khủng:** Đây chính là nền tảng cốt lõi của ngôn ngữ lập trình trí tuệ nhân tạo **Prolog** (Programming in Logic) và các **Hệ**

chuyên gia (Expert Systems).

Mô phỏng Liên kết Ngược

function FOL-BC-ASK($KB, query$) **returns** a generator of substitutions
return FOL-BC-OR($KB, query, \{\}$)

function FOL-BC-OR($KB, goal, \theta$) **returns** a substitution
for each $rule$ **in** FETCH-RULES-FOR-GOAL($KB, goal$) **do**
 $(lhs \Rightarrow rhs) \leftarrow$ STANDARDIZE-VARIABLES($rule$)
 for each θ' **in** FOL-BC-AND($KB, lhs, UNIFY(rhs, goal, \theta)$) **do**
 yield θ'

function FOL-BC-AND($KB, goals, \theta$) **returns** a substitution
if $\theta = failure$ **then return**
else if LENGTH($goals$) = 0 **then yield** θ
else
 $first, rest \leftarrow$ FIRST($goals$), REST($goals$)
 for each θ' **in** FOL-BC-OR($KB, SUBST(\theta, first), \theta$) **do**
 for each θ'' **in** FOL-BC-AND($KB, rest, \theta'$) **do**
 yield θ''

Figure 3: Liên kết Ngược: Bắt đầu từ kết luận, lùi ngược dòng tìm kiếm chứng cứ xác thực (Prolog hoạt động hoàn toàn theo cách này).

9.5 Phân giải FOL (Resolution)

◇ Forward và Backward Chaining cực nhanh nhưng **không hoàn thiện** (Có những mệnh đề đúng nhưng chúng không thể chứng minh được nếu cấu trúc không chuẩn hóa).

◇ **Thuật toán Phân giải (Resolution) trong FOL:** Là công cụ suy diễn tổng quát và **hoàn thiện (Complete)** nhất, bao gồm các bước:

- **Bước 1:** Đưa câu FOL về dạng chuẩn CNF (Khử $\Rightarrow$, khử $\forall$, Skolem hóa để khử $\exists$).
- **Bước 2:** Dùng kĩ thuật **Hợp nhất (Unification)** để triệt tiêu các thành phần ngược dấu. (VD: Triệt tiêu $P(x)$ và $\neg P(\text{John})$ bằng phép thế $\{x/\text{John}\}$).
- **Bước 3:** Chứng minh bằng Phản chứng. Nếu sinh ra mệnh đề rỗng, bài toán được chứng minh!

Chuẩn hóa về CNF (Skolemization)

◇ Khử lượng từ $\exists$ trong FOL rất tinh vi.

◇ **Skolem hóa (Skolemization)**: Nếu lượng từ $\exists$ nằm bên trong lượng từ $\forall$, ta không thể thay $\exists$ bằng một hằng số cố định, mà phải thay bằng một **Hàm Skolem**.

- "*Mọi người đều có một trái tim*":
 $\forall x \text{ Person}(x) \Rightarrow \exists y \text{ Heart}(y) \wedge \text{Has}(x, y)$
- Ta không thể thay y bằng H_1 (Vì thế hóa ra mọi người đều xài chung 1 quả tim H_1 sao?).
- Ta phải thay y bằng hàm $F(x)$: Trái tim F **phụ thuộc** vào người x .
 $\Rightarrow \text{Person}(x) \Rightarrow \text{Heart}(F(x)) \wedge \text{Has}(x, F(x))$

Mô phỏng Phân giải FOL

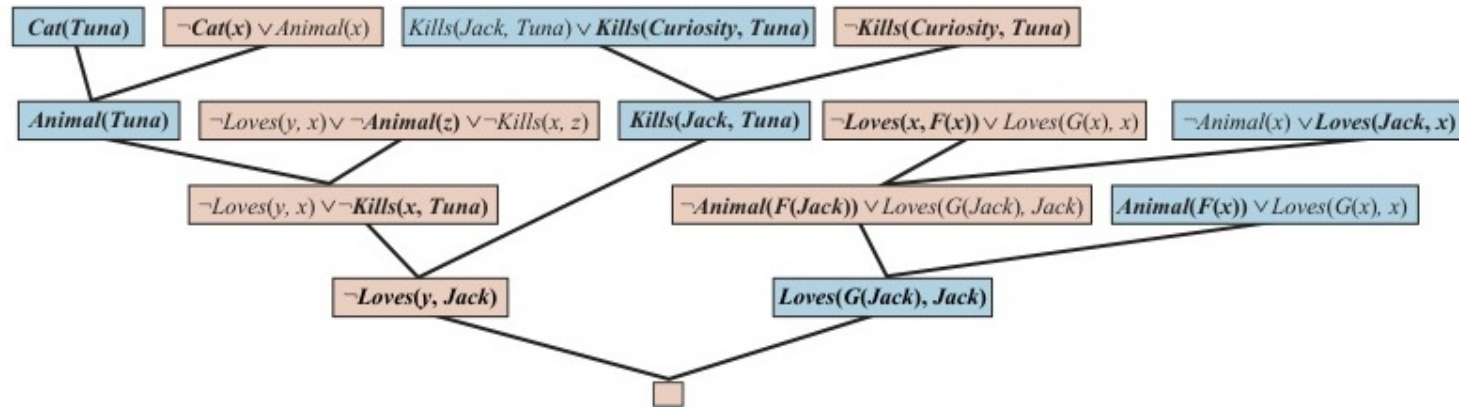


Figure 4: Cây Phân giải FOL: Các mệnh đề được hợp nhất và triệt tiêu nhau, kèm theo phép thế biến để truy vết ra đáp án.

Tóm tắt Chương 9

- ◇ **Hợp nhất (Unification):** Là trái tim của suy diễn FOL, giúp máy tính biết cách "khớp" các biến với nhau để tìm ra chính xác đối tượng, thay vì phải tính toán mù quáng hàng triệu phương trình mệnh đề.
- ◇ **Liên kết Thuận (Forward Chaining):** Cơ máy suy luận hướng dữ liệu, đẩy dữ kiện từ hiện tại đi tới tương lai. (Phù hợp với các hệ thống giám sát thời gian thực).
- ◇ **Liên kết Ngược (Backward Chaining):** Cơ máy suy luận hướng mục tiêu, lội ngược dòng từ đích để truy tìm bằng chứng. (Là cốt lõi của Prolog và Hệ chuyên gia chẩn đoán).
- ◇ **Phân giải (Resolution) & Skolem hóa:** Công cụ tổng quát vô định, kết hợp chuẩn hóa CNF và chứng minh bằng phản chứng để giải quyết bất kỳ bài toán logic tự động nào.